Geometric Motion Planning

Theory and Algorithms

Travis Haran

August 29, 2026

Contents

1 Introduction

This document contains the theoretical foundations behind the Geometric Motion Planning project.

The goal of the project is to develop a motion-planning framework from first principles, beginning with fundamental computational geometry operations and gradually progressing toward classical robot motion planning, configuration-space methods, sampling-based planning, and humanoid footstep planning.

The theory presented here is developed alongside the implementation. Each major algorithm includes its mathematical foundation, geometric interpretation, computational complexity, and connection to the C++ implementation.

2 Geometric Primitives

The motion-planning system begins with a small collection of geometric primitives. These structures provide the representation used by all later geometry and planning algorithms.

2.1 Point

A two-dimensional point is represented as

$$p = (x, y), \qquad x, y \in \mathbb{R}.$$

In the implementation, coordinates are stored using double-precision floating-point values. A point is represented in C++ as:

```
struct Point {
    double x;
    double y;
};
```

2.2 Line Segment

A line segment is defined by two endpoints A and B and is denoted by $\overline{AB}$. If

$$A = (x_A, y_A), \qquad B = (x_B, y_B),$$

then the segment contains every point

$$P(t) = A + t(B - A), \qquad 0 \le t \le 1.$$

The C++ representation is:

```
struct Segment {
    Point a;
    Point b;
};
```

2.3 Polygon

A polygon is represented as an ordered sequence of vertices

$$\mathcal{P} = \{p_0, p_1, \ldots, p_{n-1}\}.$$

Each consecutive pair of vertices forms an edge, and the final vertex connects back to the first. Thus the edges are

$$\left(p_i, p_{(i+1) \bmod n}\right), \qquad 0 \le i < n.$$

This modular indexing automatically generates the closing edge $p_{n-1} \to p_0$.

For this project, a valid polygon contains at least three vertices. Polygons with fewer than three vertices are treated as invalid and therefore produce no polygon edges.

Obstacles are assumed to be *simple polygons* unless otherwise stated. A simple polygon has no self-intersections between non-adjacent edges. Self-intersecting polygons are outside the current geometric model.

2.4 Path

A robot path is represented using an ordered list of waypoints

$$P_0, P_1, \ldots, P_k.$$

The corresponding path segments are

$$(P_0, P_1), (P_1, P_2), \ldots, (P_{k-1}, P_k).$$

Storing waypoints rather than explicitly storing every segment avoids duplicating information and simplifies path modification.

3 Orientation and the 2D Cross Product

One of the most important predicates in computational geometry is the orientation test. Given three points

$$A = (x_A, y_A), \qquad B = (x_B, y_B), \qquad C = (x_C, y_C),$$

we want to determine whether traveling from A to B and then toward C represents a left turn, right turn, or no turn.

Define

$$\overrightarrow{AB} = B - A, \qquad \overrightarrow{AC} = C - A.$$

The two-dimensional cross product is

$$\overrightarrow{AB} \times \overrightarrow{AC} = (x_B - x_A)(y_C - y_A) - (y_B - y_A)(x_C - x_A).$$

We therefore define

$$\text{orientation}(A, B, C) = (x_B - x_A)(y_C - y_A) - (y_B - y_A)(x_C - x_A).$$

The sign gives the geometric relationship:

$$\text{orientation}(A, B, C) \begin{cases} > 0, & \text{counterclockwise / left turn,} \\ < 0, & \text{clockwise / right turn,} \\ = 0, & \text{collinear.} \end{cases}$$

3.1 Geometric Interpretation

The magnitude of the cross product is the area of the parallelogram formed by $\overrightarrow{AB}$ and $\overrightarrow{AC}$. Therefore the area of triangle ABC is

$$\text{Area}(ABC) = \frac{1}{2} \left|\text{orientation}(A, B, C)\right|.$$

If the orientation is zero, the triangle has zero area, meaning that the three points are collinear.

3.2 Floating-Point Considerations

When coordinates are represented using floating-point values, exact comparison with zero can be unreliable. Instead, a small tolerance ε is used:

$$|x| < \varepsilon.$$

In this project,

$$\varepsilon = 10^{-9}.$$

Values sufficiently close to zero are therefore treated as zero when checking collinearity.

This tolerance improves the behavior of interactive geometric tests, but it does not make the geometry kernel an exact-arithmetic or fully robust computational-geometry implementation. In particular, some orientation sign tests still use ordinary floating-point comparisons. The current predicates are intended for motion-planning experiments rather than numerically pathological inputs. More sophisticated robust predicates can be introduced later if they become necessary.

4 Segment Intersection

Given two line segments $\overline{AB}$ and $\overline{CD}$, we want to determine whether they intersect. This operation is fundamental to collision detection, polygon validation, visibility testing, and motion planning.

4.1 Proper Intersections

A proper intersection occurs when two line segments cross through the interiors of both segments.

Let the two segments be

$$\overline{AB} \qquad \text{and} \qquad \overline{CD}.$$

Define

$$o_1 = \text{orientation}(A, B, C),$$

$$o_2 = \text{orientation}(A, B, D),$$

$$o_3 = \text{orientation}(C, D, A),$$

and

$$o_4 = \text{orientation}(C, D, B).$$

The segments properly intersect when the endpoints of each segment lie on opposite sides of the other segment:

$$o_1 o_2 < 0$$

and

$$o_3 o_4 < 0.$$

A proper intersection therefore excludes endpoint touching and collinear overlap. These cases may still count as general segment intersections, but they are geometrically different from a crossing through both segment interiors.

This distinction is useful for visibility testing. A candidate visibility edge should be rejected when it properly crosses an obstacle boundary, while some forms of boundary touching must remain allowed.

4.2 Degenerate Cases

The proper-intersection test alone does not handle endpoint touching, collinear overlap, or one endpoint lying on another segment. These cases require a point-on-segment test.

A point P lies on $\overline{AB}$ if

$$\text{orientation}(A, B, P) = 0$$

and

$$\min(x_A, x_B) \leq x_P \leq \max(x_A, x_B),$$
$$\min(y_A, y_B) \leq y_P \leq \max(y_A, y_B).$$

In the current project convention, endpoint touching and collinear overlap both count as intersection.

4.3 Complexity

The segment-intersection test performs a constant number of arithmetic operations. Therefore,

$$T(n) = O(1).$$

Although simple, this predicate becomes one of the most frequently used operations in later motion-planning algorithms.

5 Point-in-Polygon Testing

Given a polygon $\mathcal{P}$ and query point q, we want to determine whether q lies inside the polygon. The implementation in this project uses the winding-number algorithm.

5.1 Winding Number

Imagine tracing the polygon boundary while observing how many times the polygon winds around the query point. For a simple polygon,

$$WN(q) = \begin{cases} 0, & \text{if } q \text{ is outside,} \\ +1 \text{ or } -1, & \text{if } q \text{ is inside.} \end{cases}$$

The sign depends on whether the polygon vertices are ordered counterclockwise or clockwise. The implementation therefore uses

$$WN(q) \neq 0$$

as the inside condition.

5.2 Upward Crossings

For an edge from A to B, an upward crossing occurs when

$$y_A \leq y_q \qquad \text{and} \qquad y_B > y_q.$$

If additionally

$$\text{orientation}(A, B, q) > 0,$$

the winding number is incremented:

$$WN \leftarrow WN + 1.$$

5.3 Downward Crossings

A downward crossing occurs when

$$y_B \leq y_q \qquad \text{and} \qquad y_A > y_q.$$

If

$$\text{orientation}(A, B, q) < 0,$$

then

$$WN \leftarrow WN - 1.$$

The asymmetric inequalities prevent polygon vertices lying exactly on the horizontal query line from being counted twice.

5.4 Boundary Convention

For collision detection, this project treats points lying on the polygon boundary as inside the obstacle. Therefore, before updating the winding number, each edge is checked using the point-on-segment predicate.

5.5 Boundary and Strict Interior

The Phase 1 point-in-polygon predicate treats a point on the polygon boundary as inside the polygon. This convention is useful for collision checking because touching an obstacle is considered a collision.

Visibility testing requires a finer distinction. A point may be:

- strictly inside the polygon;
- on the polygon boundary; or
- outside the polygon.

We therefore distinguish between the existing point-in-polygon relation and strict interior membership.

A point p is strictly inside a polygon P when

$$p \in \text{Interior}(P).$$

A point lying on an edge or vertex belongs to the boundary,

$$p \in \partial P,$$

and is therefore not considered strictly inside.

This distinction will become important when constructing visibility graphs, because a visibility segment may terminate at or travel along an obstacle boundary without entering the obstacle interior.

5.6 Complexity

Each polygon edge is examined exactly once. For a polygon containing n vertices,

$$T(n) = O(n).$$

6 Collision Checking

The geometric predicates developed so far can now be combined to answer questions relevant to robot motion.

6.1 Segment–Polygon Boundary Intersection

Given a segment s and polygon $\mathcal{P}$, the segment is tested against every polygon edge. If

$$E(\mathcal{P}) = \{e_0, e_1, \ldots, e_{n-1}\},$$

then we evaluate

$$\text{segmentsIntersect}(s, e_i)$$

for every edge e_i.

The segment intersects the polygon boundary if

$$\exists e_i \in E(\mathcal{P}) \text{ such that } s \cap e_i \neq \varnothing.$$

For a polygon containing n edges, this requires $O(n)$ segment-intersection tests.

6.2 Why Boundary Intersection Is Not Enough

Consider a segment whose two endpoints lie completely inside a polygon. The segment may never cross the polygon boundary. In that case,

$$\text{boundary intersection} = \text{false},$$

even though the segment lies entirely inside the obstacle.

This demonstrates an important distinction between *boundary intersection* and *collision*.

6.3 Collision-Free Segment

A segment is considered collision free only if, for every obstacle,

1. the segment does not intersect the obstacle boundary,
2. the first endpoint is not inside the obstacle, and
3. the second endpoint is not inside the obstacle.

Symbolically,

$$\text{CollisionFree}(s) = \neg\big(\text{BoundaryIntersection}(s) \vee \text{Inside}(s.a) \vee \text{Inside}(s.b)\big).$$

Touching an obstacle boundary is currently considered a collision.

6.4 Collision-Free Path

A path consisting of waypoints

$$P_0, P_1, \ldots, P_k$$

contains the segments

$$(P_i, P_{i+1}), \qquad 0 \le i < k.$$

The path is collision free if every one of these segments is collision free:

$$\text{CollisionFreePath}(\mathcal{P}) = \bigwedge_{i=0}^{k-1} \text{CollisionFree}\big(\overline{P_i P_{i+1}}\big)\,.$$

An empty path contains no motion segments and is treated as collision free by the current representation. A one-waypoint path is a stationary configuration: it is collision free only when that waypoint does not lie inside or on the boundary of an obstacle.

Demo 1C visualizes this path-level predicate. The entire path is drawn green when every segment is collision free and red when any part of the path is invalid. This emphasizes the distinction between local segment validity and whole-path validity.

6.5 Phase 1 Conventions and Assumptions

The geometry kernel developed in Phase 1 uses the following conventions:

- obstacle polygons are simple polygons with at least three vertices;
- self-intersecting polygons are outside the current model;
- a point on an obstacle boundary is considered in collision;
- a segment that touches an obstacle edge or vertex is considered in collision; and
- collinearity uses a small floating-point tolerance rather than exact arithmetic.

The implementation is deliberately layered. Higher-level motion-validity queries are built from lower-level geometric predicates:

$$\text{orientation} \longrightarrow \text{segmentIntersection} \longrightarrow \text{polygonIntersection} \longrightarrow \text{segmentCollision} \longrightarrow \text{pathCollision}\,.$$

This creates the first complete geometric motion-validity test in the project. Later planners will build on these predicates when constructing visibility graphs, PRMs, RRTs, and humanoid footstep transitions.

6.6 Transition to Visibility Graphs

Phase 2 introduces an important distinction between *collision-free motion* and *geometric visibility*. The current collision convention rejects any segment that touches an obstacle boundary. This behavior should remain unchanged for ordinary motion validation.

A visibility graph, however, uses obstacle vertices as graph nodes. A candidate visibility edge may therefore need to terminate exactly at an obstacle vertex without passing through the obstacle interior. Consequently, the ordinary collision-free-segment predicate cannot simply be weakened or reused without additional visibility-specific reasoning. Phase 2 will introduce a separate visibility predicate that preserves the Phase 1 collision semantics while handling valid obstacle-vertex endpoints.

7 Visibility Graphs

A visibility graph converts a continuous geometric planning problem into a discrete graph-search problem.

For polygonal obstacles, the graph will eventually contain the start point, goal point, and obstacle vertices as nodes. Two nodes are connected when they are geometrically visible to one another.

7.1 Geometric Visibility

Two points A and B are considered visible when the open segment between them does not pass through the interior of any obstacle.

For an obstacle O, this can be expressed as

$$(A, B) \cap \text{Interior}(O) = \varnothing.$$

The use of the open segment (A, B) is important. A visibility segment may terminate exactly at an obstacle vertex without entering the obstacle interior.

Visibility therefore differs from the Phase 1 collision convention. In ordinary collision checking, touching an obstacle boundary counts as a collision. In visibility testing, touching or traveling along an obstacle boundary may be allowed as long as the segment does not enter the obstacle interior.

7.2 Proper Boundary Crossings

A candidate visibility segment is invalid if it properly intersects an obstacle edge. A proper intersection crosses through the interiors of both segments rather than merely touching at an endpoint.

Proper-intersection testing detects ordinary crossings through an obstacle, but it is not sufficient by itself. A candidate segment can enter an obstacle through one of its vertices without producing a proper intersection.

7.3 Parametric Representation of a Segment

A point along the segment from A to B can be represented parametrically as

$$P(t) = A + t(B - A), \qquad 0 \leq t \leq 1.$$

The parameter t describes the position along the segment:

$$P(0) = A,$$

$$P(1) = B,$$

and

$$P\left(\frac{1}{2}\right) = \frac{A+B}{2}.$$

If a point V is known to lie on the nonzero segment $\overline{AB}$, its parameter can be recovered from either coordinate. For example,

$$t = \frac{V_x - A_x}{B_x - A_x}$$

when the x component is suitable for division.

The implementation uses the coordinate with the larger absolute change between A and B. This avoids division by zero for vertical or horizontal segments and reduces division by very small direction components.

The helper that computes this parameter assumes that the point already lies on a nonzero segment. It does not perform segment-membership validation.

7.4 Visibility Near Obstacle Vertices

Proper-intersection testing alone cannot detect every invalid visibility segment. In particular, a segment may pass through an obstacle vertex and immediately enter the obstacle interior.

Suppose an obstacle vertex V lies on a candidate segment and has parameter t_V. Points immediately before and after the vertex can be sampled using

$$P(t_V - \delta)$$

and

$$P(t_V + \delta),$$

where δ is a small positive parameter displacement.

If either valid neighboring sample lies strictly inside the obstacle, then the candidate segment enters the obstacle interior and is therefore not visible.

The endpoint cases require special treatment. If

$$t_V = 0,$$

then $V = A$, so no portion of the candidate segment exists before the vertex. Only the sample after the vertex needs to be considered. Similarly, if

$$t_V = 1,$$

then $V = B$, so only the sample before the vertex needs to be considered.

The sampled parameters are also clamped to the valid segment interval

$$0 \leq t \leq 1.$$

This ensures that visibility testing considers only the finite candidate segment rather than points on the infinite line extending beyond its endpoints.

7.5 Visibility Graph Construction

Given a start point S, a goal point G, and a collection of polygonal obstacles, the visibility graph is represented as

$$\mathcal{G} = (V, E).$$

The node set contains the start point, goal point, and every obstacle vertex:

$$V = \{S, G\} \cup \bigcup_{O \in \mathcal{O}} \text{Vertices}(O).$$

In the current implementation, the start point is stored as node 0, the goal point as node 1, and obstacle vertices are appended afterward.

To construct the edge set, every unique pair of nodes (v_i, v_j) with

$$i < j$$

is tested for geometric visibility. If the two nodes are visible, an undirected edge is added between them.

Thus,

$$E = \{(v_i, v_j) \mid i < j \text{ and } \text{Visible}(v_i, v_j)\}.$$

Using $i < j$ ensures that each unordered pair is considered exactly once. It also prevents self-edges such as (v_i, v_i).

7.6 Euclidean Edge Weights

Each visibility edge is assigned a weight equal to the Euclidean distance between its endpoints.

For points

$$A = (x_A, y_A) \qquad \text{and} \qquad B = (x_B, y_B),$$

the edge weight is

$$w(A, B) = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}.$$

These weights represent the geometric length of traveling directly between two visible nodes.

The resulting weighted graph can later be searched using shortest-path algorithms such as Dijkstra's algorithm or A*.

8 Shortest Paths with Dijkstra's Algorithm

Once the weighted visibility graph has been constructed, motion planning becomes a shortest-path problem on a graph.

For each node v, Dijkstra's algorithm maintains a tentative distance

$$d(v),$$

representing the shortest currently known distance from the start node to v.

Initially,

$$d(S) = 0,$$

while every other node is assigned

$$d(v) = \infty.$$

The central operation in Dijkstra's algorithm is *edge relaxation*. For an edge from u to v with weight $w(u, v)$, a new candidate distance to v is

$$d(u) + w(u, v).$$

If

$$d(u) + w(u, v) < d(v),$$

then a shorter route to v has been discovered, and we update

$$d(v) = d(u) + w(u, v).$$

The predecessor of v is also recorded as u so that the final shortest path can later be reconstructed.

8.1 Selecting the Next Node

After relaxing the edges of the current node, Dijkstra's algorithm selects the unvisited node with the smallest tentative distance.

Once a node is selected in this way, its shortest-path distance is considered finalized.

This property relies on all graph edge weights being non-negative. In the visibility graph, edge weights are Euclidean distances, so this condition is automatically satisfied.

8.2 Path Reconstruction

Whenever relaxation improves the tentative distance to a node v, the algorithm records the node u from which that improvement came.

This node is called the *predecessor* or *parent* of v.

After the goal is reached, the shortest path is reconstructed by following the parent links backward:

$$G \to \text{parent}(G) \to \text{parent}(\text{parent}(G)) \to \cdots \to S.$$

Because this produces the path in reverse order, the resulting sequence is reversed to obtain the final path from start to goal.

8.3 Undirected Visibility Edges

The visibility graph is undirected: if node u can see node v, then node v can also see node u.

For efficiency, each undirected edge is stored only once in the graph.

Therefore, when processing a node, Dijkstra's algorithm must check whether the current node appears as either endpoint of an edge.